

Script-Aware Fusion for Cross-Lingual Retrieval: Why Naive Hybrid Retrieval Fails on Indic and Code-Mixed Queries

Dhruv

School of Computer Science Engineering and Technology, Bennett University

CSET 346 — Natural Language Processing

Demo: indicragqa.vercel.app · Code: github.com/dhruvv1402/IndicRAG-QA

ABSTRACT

Retrieval-augmented generation over multilingual corpora is commonly built by fusing a lexical retriever with a dense one, on the assumption that their errors are complementary. We show that this assumption fails in a specific and correctable way when a query and its evidence are written in different scripts. A lexical retriever cannot return a passage whose script differs from the query's, because the two share almost no tokens; under rank fusion such a passage collects one vote where a same-script passage collects two, and is penalised for the lexical retriever's *blindness* rather than for its own irrelevance. On a bilingual Hindi–English corpus of Indian government welfare schemes and a sealed test split of 216 answerable questions, Reciprocal Rank Fusion reaches 0.274 Recall@5 on cross-lingual queries against 0.661 for its own dense component. We propose **script-aware fusion**, which scores each candidate over the retrievers *eligible* to return it: it recovers 0.685 cross-lingual Recall@5 (+0.411 over plain RRF, $p = 0.0001$) at no monolingual cost, though its gain over the dense retriever alone is small (+0.030 overall) and significant only on code-mixed queries. For abstention, the BM25 top score separates answerable from unanswerable questions (AUC 0.791) while the fused score does not, and it catches only half of near-miss questions. We release a 393-item bilingual QA set with a query-language $\times$ evidence-language matrix and a four-class unanswerable taxonomy, verified in two language-model passes and then, on its test split, reviewed by a person.

I. INTRODUCTION

A. The access gap

Public-interest information in India is routinely published in one language and asked about in another. A citizen checking whether their household qualifies for a post-matric scholarship is reading a circular issued in English, or its Hindi counterpart published separately and not as a translation, and is asking the question in Hindi — or, far more commonly in practice, in

Romanized Hindi–English code-mix: *"Scholarship ke liye minimum eligibility kya hai?"*

Keyword search fails on this query three times over. The language differs from the document's. The script differs from the document's. And the surface words differ from both, because Romanized Hindi has no standardised orthography, so the same word reaches the index as *eligibility*, *yogyata* and *योग्यता* depending on who typed it. Each of these is individually well studied. Their conjunction is the ordinary case for a large population of users, and it is the setting this paper measures.

The stakes are not those of open-domain question answering. Documents that govern who receives public money are consulted by people deciding whether to apply, and a confident wrong answer about an eligibility ceiling or a deadline is materially worse than no answer at all. A system in this setting must be able to say that it does not know, and must be evaluated on whether it says so in the right places rather than only on the quality of the answers it does give.

B. The standard remedy, and where it breaks

The accepted response to the language and script mismatch is multilingual dense retrieval, which embeds queries and passages into a shared space and so is not bound by surface overlap. The accepted response to dense retrieval's weakness on rare terms — scheme names, amounts, section numbers — is hybrid retrieval: combine the dense retriever with a lexical one and fuse their rankings, most commonly with Reciprocal Rank Fusion, which requires no score normalisation and is strong enough to be a standard baseline.

We find that hybrid retrieval assembled in this standard way does not merely fail to help the cross-lingual case. It *actively harms* it, scoring below the dense component it contains. On the verified test split of our corpus, plain RRF reaches 0.274 Recall@5 on cross-lingual queries while its own dense retriever, used alone, reaches 0.661.

The cause is structural rather than a matter of tuning, which is why an α sweep over weighted fusion found nothing and reported a clean negative result. Rank fusion treats agreement between retrievers as evidence and therefore discounts a

candidate returned by only one of them. Across a script boundary the lexical retriever has zero recall *by construction* — it cannot return a Devanagari passage for a Latin-script query at any rank, for any query, however relevant — so its silence carries no information. Fusion nonetheless charges the candidate for that silence, and does so precisely in the cases where the dense retriever is the only informative signal available.

The correction that follows from this diagnosis is small: score each candidate over the retrievers that were *eligible* to return it rather than over those that did. It requires no training, no tuning and one lookup per candidate, and it recovers cross-lingual Recall@5 from 0.274 to 0.685 ($p = 0.0001$) with no measurable cost on monolingual queries (0.964 against 0.964). What it does not do is beat the dense retriever alone by much: +0.030 overall ($p = 0.052$), and significantly only on code-mixed queries (+0.085, $p = 0.004$). The contribution is removing a harm that standard fusion introduces, not a large gain over a strong dense baseline, and we report it as such.

C. How the result was found

We note the path deliberately, because it bears on how such systems should be evaluated. The aggregate retrieval tables showed hybrid fusion underperforming and gave no indication why; every component behaved as specified, and the hyperparameter sweep returned a negative result about a question that was not the relevant one. The mechanism became visible only on a structured reading of twenty failure cases, in which the dense retriever alone had found the gold passage in eight and plain RRF had lost all eight. A fusion method that discards every case its stronger component got right is not a method with a disappointing coefficient. Aggregate metrics did not merely fail to show this; they pointed away from it.

D. Contributions

- **Script-aware fusion.** A correction to rank fusion for retrievers with asymmetric coverage of a collection, with its structural cause, a paired significance test, and its cost reported alongside its benefit. We state the scope precisely: RRF's discount on single-retriever evidence is intended behaviour rather than an oversight, and prior multilingual retrieval work already avoids the problem by restricting sparse retrieval to the query's language. Our contribution is to show that the fusion *arithmetic* still treats a retriever's silence as evidence, to quantify what that costs, and to correct it in a form that generalises beyond language to any asymmetric-coverage setting.
- **A bilingual evidence-grounded QA set.** 393 items over Indian government welfare schemes, with an explicit query-language $\times$ evidence-language matrix covering six

cells, a four-class unanswerable taxonomy, per-item gold passage sets, and a sealed dev/test split. It was verified in two independent language-model passes, and every item of the test split was then reviewed by a person; every report built from it says which items were checked by whom (§III-D, §IX).

- **Which retrieval score carries an abstention signal.** On verified questions the BM25 top score separates answerable from unanswerable (medians 25.5 against 13.8, AUC 0.791; test F1 0.525), while the rank-fused score does not (0.0328 against 0.0325), because rank fusion discards magnitudes. A threshold on the lexical score rejects every out-of-scope question and about half of near-misses — questions about a covered scheme asking for a fact it does not state — which only a reader of the passage can catch. Before verification the same measurement showed no signal at all; malformed questions had hidden it, and we report both.
- **A controlled negative result on Indic MLM checkpoints.** Off-the-shelf masked-language-model encoders for Indic languages underperform sentence-trained multilingual encoders at retrieval, with the mean-pooling confound controlled rather than left as an explanation.

E. Organisation

Section II surveys cross-lingual retrieval, code-mixed IR and rank fusion. Section III describes the corpus and the QA set. Section IV describes the system, Section V the experimental protocol, and Section VI the results. Section VII presents the error analysis from which the main result came, Section VIII discusses the mechanism and where it generalises, and Sections IX and X state limitations and conclusions.

II. RELATED WORK

Our work sits at the intersection of four lines of research.

A. Multilingual dense retrieval

Dense retrieval over multilingual encoders is the standard remedy for the vocabulary and language mismatch that defeats lexical search. Sentence-level encoders trained with a contrastive or retrieval objective — LaBSE (Feng, Yang, Cer, Arivazhagan and Wang, ACL 2022, 878–891), multilingual E5 (Wang et al., arXiv:2402.05672, 2024), and multilingual Sentence-BERT (Reimers and Gurevych, EMNLP-IJCNLP 2019, with the multilingual extension by knowledge distillation, EMNLP 2020) — project queries and passages into a shared space in which translation-equivalent texts are near neighbours.

Benchmarks for this setting are predominantly *monolingual-per-language* or cross-lingual at the language rather than the script level. Mr. TyDi (Zhang et al., arXiv:2108.08787) established a multilingual dense-retrieval benchmark across eleven typologically diverse languages; the MIA 2022 shared task (Asai et al., arXiv:2207.00758) extended open-retrieval QA to sixteen languages; and AfriQA (Ogundepo et al., arXiv:2305.06897) covers cross-lingual open-retrieval QA for African languages. These establish that multilingual dense retrieval works and that its quality varies sharply by language. What they do not isolate is the interaction we study: what happens to a *fused* system when the query script and the evidence script differ.

B. Indic language representations

Encoders pretrained specifically on Indian languages are the natural first choice for this corpus. MuRIL (Khanuja et al., arXiv:2103.10730, 2021) is trained on Indian-language corpora and, notably for our purposes, augments pretraining with **translated and transliterated document pairs** to supply explicit cross-lingual signal; it outperforms mBERT across the XTREME benchmark. IndicBERT and the wider AI4Bharat resources (Kakwani et al., IndicNLPSuite, Findings of EMNLP 2020) serve a similar role.

The gap we address is that these models are evaluated on classification and understanding benchmarks, and their readiness for *retrieval* off the shelf is generally assumed rather than measured. The distinction is not incidental. Both MuRIL and IndicBERT are masked-language-model checkpoints with no pooling layer trained for sentence representation, and MLM representations are known to be anisotropic — occupying a narrow cone in which the average cosine similarity of unrelated points is high — which is precisely the geometry that defeats similarity search (Li et al., EMNLP 2020, arXiv:2011.05864). This anisotropy has been shown to persist in multilingual models for cross-lingual semantic similarity specifically (arXiv:2306.00458). Our §VI-B result quantifies the consequence for Indic retrieval, and is sharpened by MuRIL's transliteration-pair pretraining: the model given explicit transliteration signal is the one that retrieves nothing on our code-mixed slice.

C. Hybrid and fused retrieval

Combining a lexical and a dense retriever is standard practice, on the premise that their errors are complementary: lexical matching is precise on rare terms, names and numbers, while dense retrieval generalises over paraphrase and synonymy. Reciprocal Rank Fusion (Cormack, Clarke and Büttcher, SIGIR 2009, pp. 758–759) remains the default combiner, chosen for needing no score normalisation and no

tuning. Neural retrieval has since produced alternatives to the sparse half of the pair — ColBERT's late interaction over contextualised token embeddings (Khattab and Zaharia, SIGIR 2020, 39–48) and SPLADE's learned sparse representations with term expansion (Formal, Piwowarski and Clinchant, SIGIR 2021) — but neither removes the constraint that concerns us. SPLADE in particular expands terms within its model vocabulary, which mitigates vocabulary mismatch but not *script* mismatch: expansion cannot bridge a Devanagari passage and a Latin query that share no surface form.

RRF rewards documents ranked highly by several retrievers and, by construction, penalises a document returned by only one. This is the intended semantics and is well founded **when every retriever is capable of returning every document** — a condition that holds on the monolingual collections where fusion was developed and validated. Our §VI-E result concerns what happens when it does not.

Tu and Padmanabhan (MIA 2022 shared task submission, arXiv:2207.01940) come closest to the problem, restricting the sparse index to the question's language in a dense-sparse hybrid for cross-lingual QA. That addresses the retrieval step; the fusion step still treats a dense-only passage as weakly evidenced.

D. Answerability and grounded abstention

Systems that answer from retrieved evidence must be able to decline. SQuAD 2.0 (Rajpurkar, Jia and Liang, ACL 2018, 784–789) established unanswerable questions as a first-class evaluation target, and did so adversarially: its 50,000+ unanswerable questions were written by crowdworkers *to resemble answerable ones*. Confidence-based abstention for retrieval-augmented systems is an active area, including risk control for RAG (Findings of EMNLP 2024) and uncertainty-aware evidence verification [VERIFY – pick one or two current citations rather than gesturing at a literature].

Our claim here must be stated carefully, because SQuAD 2.0 partly anticipates it. Adversarially authored unanswerable questions are already near-misses in spirit. What the existing resources do not do is *stratify* them, so results are reported over an undifferentiated unanswerable class. Our §III-E taxonomy separates four kinds, and §VI-H shows why the distinction is not cosmetic: 55 of our 80 unanswerable items concern schemes that *are* in the corpus. A BM25 threshold rejects every out-of-scope question on our test split and only 0.522 of near-misses, whose passages are genuinely about the right scheme; a single F1 over an undifferentiated unanswerable class hides that. The contribution is the stratified measurement, not the observation that hard unanswerable questions exist.

E. Code-mixed retrieval

Romanized Hindi-English is not a marginal input mode. **The published platform-level statistics conflict, and the paper should say so rather than pick the most favourable.** Figures in the deromanization literature put 93.17% of native Hindi speakers' social-media posts in Roman script against 2.93% in Devanagari, and a 2015 YouTube analysis reports 52% against 1%; yet another measurement reports Devanagari on Twitter *rising* from 35% in 2014 to 82% in

- These are not reconcilable as stated and likely reflect different platforms, sampling frames and dates.

The claim we can make safely is the code-mixing trend rather than the script split: a peer-reviewed study of Indian Twitter users reports the proportion preferring Hinglish rising from 44.9% in 2014 to 56.3% after 2020 (*Humanities and Social Sciences Communications*, 2024, DOI 10.1057/s41599-024-03058-6) [VERIFY – title and authors confirmed via search; the article is behind an authentication redirect, so confirm the figures against the full text before quoting them].

Work on code-mixed text has concentrated on language identification, transliteration and deromanization as preprocessing, typically to normalise input into a single native script before a downstream monolingual model. That framing treats script mismatch as a preprocessing problem to be eliminated. We instead retain it as an experimental variable, because the query script is exactly what determines whether a lexical retriever can see a passage at all — and therefore whether fusion helps or harms.

III. DATASET

A. Corpus construction

We assemble a parallel Hindi–English corpus of Indian government welfare and education schemes. The corpus is bilingual *by construction* rather than by translation: each scheme is represented by two independently authored documents, one in each language, so that a query in one language may legitimately be answered by evidence in the other. This is what makes cross-lingual retrieval a measurement here rather than a simulation.

Scheme pairs are resolved through the MediaWiki langlinks API. We seed a list of English article titles covering education, skills, financial inclusion, pensions, social welfare, housing, health, agriculture and governance, and retain only those for which Wikipedia itself asserts a Hindi counterpart. Parallelism is therefore an editorial claim by the source, not an assumption by us. Of 49 seeded schemes, 40 yielded a usable pair; the remainder either had no Hindi article or had one

shorter than 500 characters, and stub articles were rejected rather than padded. The final corpus is **40 schemes × 2 languages = 80 documents**, spanning 14 scheme categories.

A deviation from our original design, and the reason for it. We first attempted to build the corpus from official ministry guideline documents, which would have given regulatory rather than encyclopedic text. Two obstacles made this infeasible. First, the Hindi paths on `socialjustice.gov.in` return HTTP 200 with a "Page not found" body — a soft 404 — so the bilingual premise fails at the source. Second, and decisively, the scheme guideline PDFs are scans: the 18-page, 2.8 MB *Post-Matric Scholarship for SC Students* guidelines document yields **17 characters** of extractable text, and no OCR engine was available in our environment. A corpus of English-only text-layer documents would have eliminated the cross-lingual question the work exists to study. We consider the trade worthwhile and state its cost explicitly in §IX: our results describe retrieval over *descriptive* text about government schemes, and support no claim about retrieval over statutory or regulatory prose.

B. Devanagari integrity validation

Text extraction is the highest-risk stage of an Indic corpus pipeline, and its characteristic failure is silent. Legacy or subset-embedded fonts produce output that renders as plausible Devanagari while being nonsense at the character level: dependent vowel signs reordered ahead of their consonant, conjuncts broken at the virama, or glyphs mapped into the Latin private-use area. None of this raises an error. It produces text that a reader unfamiliar with the script will accept, and once indexed it degrades retrieval in a way that resembles poor model quality.

We therefore validate every extracted Hindi document against five independent checks, and report them because we believe the procedure is reusable:

- **Misplaced dependent vowel signs.** In a well-formed orthographic syllable a matra (U+093A–U+094C) must follow a consonant or nukta. A matra in any other position is the signature of the reordering bug.
- **Dangling word-final viramas.** A virama joins two consonants; word-final it indicates the second half of a conjunct was lost.
- **Devanagari codepoint ratio,** which catches a font mapped outside the block.
- **Hindi function-word rate.** Text can be entirely within the Devanagari block and still not be Hindi; a stopword-rate floor catches that mojibake.
- **Unicode replacement characters,** indicating an encoding failure.

All 80 documents pass. One finding from developing these checks is worth reporting for other Indic pipelines. Three clean documents initially failed the dangling-virama check, and the cause was not corruption: formal Hindi prose writes a **zero-width non-joiner after a virama** to force an explicit halant rather than a conjunct ligature, as in उद्देश्य. ZWNJ lies outside the Devanagari block, so a word-tokenizer keyed on that block splits the token and leaves its first half apparently terminating in a bare virama. Stripping zero-width characters before validation resolves it. We retain a tolerance rather than requiring zero final viramas, because Hindi does write some genuinely, such as the Sanskrit-derived adverb अर्थात्.

C. Passage segmentation

Segmentation bounds the achievable retrieval quality: a fact split across a chunk boundary cannot be retrieved intact regardless of encoder. We therefore split on document structure first and length second. Source articles carry explicit section markers, which we use both as split points and as retained metadata — a passage's section path ("Eligibility > Income criteria") is genuine context for a generator, and it distinguishes the many near-identical eligibility passages that different schemes contribute.

Sections comprising navigation or bibliography (references, external links, see also, and their Hindi equivalents) are discarded; retaining them would place link lists in the retrieval pool, where they match many queries lexically and answer none. Within a section, sentences are packed to a target of 170 tokens, bounded at 240, with 25% overlap between adjacent chunks where a split was length-driven rather than structural. Overlap is not applied across section boundaries, since those are real semantic breaks and bleeding across one would place a clause from one scheme inside another scheme's passage. **Two corrections belong here, because both were found by measuring rather than by reading.** The committed corpus contains 29 passages above the 240 bound, the largest 415 tokens. The packer respects the bound; a guard that folds an undersized chunk into the previous passage did not. It tested only that the previous passage came from the same *document*, so a short chunk from any later section merged into whatever passage happened to be last — which is precisely the bleeding across section boundaries that overlap is forbidden from doing, and it left the merged passage carrying the earlier section's path. One passage is labelled Introduction and ends in text about the Unified Payment Interface, three sections later. The guard now requires the same section and a result within the bound.

The corpus has **not** been re-segmented, and that is the second correction. Passage identifiers are positional — `scheme-lang#p0007` is the eighth passage of that document —

so they are stable only while segmentation is. Applying the fix produces 803 passages instead of 694, and although every existing identifier still resolves, **312 of them resolve to different text**, and 123 of the 320 answerable gold items cite one. A re-segmentation would therefore repoint more than a third of the gold set at passages that may no longer contain the answer, without a single broken reference to signal it. Content-derived identifiers would fail loudly instead, and are the right fix; changing the scheme now would invalidate the same gold set for the same reason. The corpus is frozen until verification completes, and the measurements in this paper are taken on the corpus as committed.

How much this costs the results. 86 passages of 694 draw sentences from more than one source section; 5 of those pair a section with its own subsection and are harmless, leaving 81 that join unrelated material, and 46 of the 320 answerable gold items cite one. A merged passage is larger than it should be, so it presents more surface for a query to match and might be expected to inflate recall. It does, slightly and not significantly: those 46 items reach Recall@5 0.674 [0.543, 0.804] under script-aware fusion against 0.591 [0.526, 0.650] for the other 274. The intervals overlap heavily, and at 14% of the set an effect of that size moves the overall figure by about a point.

We report the defect rather than the reassurance, because the reassurance is narrow. The retrieval numbers survive it. What does not survive is the claim that a passage's `section_path` is reliable context for a generator: for those 81 passages it names one of the sections present and not the others, which is the input §IV-A feeds the model as provenance.

Three further metadata defects, and why they went unseen. Auditing the remaining per-passage invariants found that `section_path` is not the only provenance field that does not mean what it says. 29 passages exceed the stated 240-token bound, one reaching 321. `text_raw`, documented as the source spelling kept alongside the normalized text so that a cited amount is displayed as the circular wrote it, is itself normalized: 691 of 694 passages have a `text_raw` that normalization leaves unchanged, and no Hindi passage retains the Devanagari digits its source uses. `char_span` is worse still - the segmenter advances its cursor by the normalized chunk length while searching raw source text, so the offsets drift monotonically within a document, and only 22.8% name a slice that recovers their own passage while 56 have drifted past the end and name nothing.

All three share the root cause the section-spanning defect has: segmentation splits sentences after normalizing a section and never keeps a mapping back to the source. They share something more instructive too. Every one of these fields is written, serialized, and documented, and *read by nothing* -- so

none of them could fail loudly, and a downstream citation view built on `char_span` would have pointed a reader at the wrong text roughly four times in five. This is the same failure mode as §VI-D’s uninstantiated NLI scorer and the unread `cited_found` flag: not incorrect code that broke, but correct-looking code that no execution path ever reached. We therefore added `corpus_audit`, which reads every one of these invariants and reports a per-check rate.

Repairing two of them without unfreezing the corpus.

The obvious fix -- re-running segmentation with a source-offset mapping -- is unavailable, because it would also apply the merge-guard correction and so change how many passages each document yields, shifting every identifier after the first merge. But the freeze protects `passage_id` and `text`, and neither is what these two fields need. `corpus_repair-spans` therefore re-derives them in place: each passage’s normalized text is aligned back to its source document and `char_span` is set to the located extent, with `text_raw` read from it. The alignment is sentence-wise rather than a single forward walk, because a passage is not always contiguous in its source -- overlap re-emits the tail of the previous chunk, so the same sentences can appear twice with the second copy sitting behind where the first ended. A single-run aligner scored 0.03 coverage on 39 passages that are in fact entirely present in their documents.

The repair refuses to write if any `passage_id`, `text` or `token_count` moved; none did, so no index, cache, gold reference or published number in this paper is affected. Span fidelity rises from 25.4% to 98.8% of the passages that carry a span, 99.1% carry one, and `text_raw` now preserves source spelling for every passage -- 21 Hindi passages recover the Devanagari digits their sources use, which is exactly the fidelity the field was introduced to provide. The six passages the aligner cannot place have their span *cleared* rather than left drifted, on the principle that an absent span is honest where a wrong one misleads; `corpus_audit` counts the two states separately so a repair cannot hide its misses among its refusals.

That separation earned its keep immediately. The first version of the aligner declined 23 passages, and inspecting them showed every sentence of 20 of them aligning perfectly when searched from the start of the document: the search retried from the top only when a forward pass returned *nothing*, so a half-matching anchor found ahead of the cursor was committed to instead of the true match behind it. Retrying whenever the forward pass does poorly, rather than only when it fails outright, took the declines from 23 to 6. What remains unfixed is the third defect: the 29 oversized passages, and the welded ones whose span must swallow a heading to cover them, both require re-chunking and so wait on the same re-anchoring as the merge fix.

The gold set, the embedding caches and the error analysis all key off these identifiers, which is what makes their apparent stability worth stating precisely rather than assuming.

This yields **694 passages** (491 English, 203 Hindi) across 40 schemes. Passage length has median 170 tokens (mean 157, interquartile range 111–193, range 20–415), and 82.7% carry a section path beyond the article lead.

The language imbalance is inherent and should be read as a property of the source. Hindi Wikipedia articles run roughly 40% the length of their English counterparts, giving a 0.41 Hindi-to-English passage ratio. Scheme coverage is also uneven, from 3 to 62 passages per scheme. Both facts constrain the dataset design in §III-D: Hindi-evidence questions are drawn from a smaller pool, and passage selection must be stratified across schemes to prevent a single long article dominating a matrix cell.

D. Question-answer set

(Verified 2026-09-23 in two language-model passes; test split reviewed by a person 2026-09-24. 393 items survive, 7 were rejected.)

The set was built as 400 instances, 320 answerable and 80 unanswerable; after verification it holds 393, 313 answerable and 80 unanswerable. The answerable items follow an explicit **query-language × evidence-language matrix** (Table I), which is what permits the monolingual / cross-lingual / code-mixed comparison in §VI-B to be made on balanced strata rather than on whatever distribution happened to arise.

TABLE I — Answerable items by query and evidence language (verified; design target in parentheses)

Query ↓ / Evidence →	English	Hindi	Total
English	69 (70)	45 (45)	114
Hindi (Devanagari)	44 (45)	59 (60)	103
Hinglish (Romanized)	51 (55)	45 (45)	96
Total	164	149	313

Three groupings follow directly: **monolingual** (128), where query and evidence share a language and script; **cross-lingual** (89), different languages each in native script; and **code-mixed** (96), Romanized Hindi-English queries whose evidence is never in the query’s script. The Hinglish row is never monolingual by construction, since Romanized Hindi is not the script of any corpus document — which is precisely why code-mixed queries are expected to be hardest.

Candidates are bootstrapped with a locally hosted quantized model (Qwen2.5-3B-Instruct, Q4_K_M). The protocol specified that a person then verify each one. **Verification instead began with a language model (Claude Opus 5.5), in two independent passes, and a person then reviewed the test split.** The first pass read every item against

its cited passage and proposed a correction or a rejection; it judged only 27 of the 320 bootstrapped answerable items correct as generated, found 63 gold answers the passage contradicts or does not state, and found 112 questions that were unreadable machine translation, almost all in the cross-lingual and Hinglish cells. The duplicated unanswerable stems (§III-E) and 7 rejected answerable items were replaced by newly written ones. A second pass, run by fresh model instances with no access to the first, then checked every item in its final form against its evidence and the passages a retriever returns for it, confirming 353 and making small corrections to 42. It rejected 5 — three because another passage in the corpus gives a conflicting figure, so the question has no single answer — and 2 more were dropped as duplicate questions. It also added, for 73 items, passages that state the same fact, most of them the other language's article (see below).

A person — the author — then reviewed every one of the 273 test-split items against its evidence, with the model passes' notes shown beside it, and accepted all 273 as they stood: no question or answer was edited and none was rejected. Two qualifications apply. The review was not blind, since the reviewer saw the model's verdict, so its 100% acceptance is a confirmation of the model passes rather than an independent labelling of the items; and it was one person. The 120 dev-split items, which are used only for fitting thresholds, were verified by the model passes alone. Each item records who checked it (annotator), and every report states the mix in its banner. The blind second pass (§IX) was run by a model instance, and its $\kappa = 1.000$ over 59 items measures one model's consistency, not agreement between annotators.

Two construction details materially affect the measurements. First, **cross-lingual items translate the question only** and leave the gold passage where it is. Translating the passage as well would convert a cross-lingual item into a monolingual one and inflate every cross-lingual figure. Second, **gold evidence is a set, not a single passage**: where a fact appears in both language versions of a scheme, either passage counts as correct. Scoring against one designated passage would penalise a cross-lingual retriever for exactly the behaviour under study.

Semantic validation of generated candidates. Bootstrapping with a small model introduces a failure mode that parse-rate monitoring cannot see. In development, a 0.5B-parameter model asked to translate इस योजना के लिए आय सीमा क्या है? into English returned "What is the salary for this job?" — well-formed JSON with a non-empty question field, and entirely invented. Such an output scores as a success under any validity check while corrupting the dataset. We therefore apply structural checks to every generated item: a translation into

Hindi must contain Devanagari and must differ from its input; a romanization must be Latin script *and* retain Hindi function words, or the model translated rather than transliterated; and a generated question must match its passage's language, not echo prompt scaffolding, and share at least one token with its own evidence. These checks cannot distinguish a good translation from a mediocre one. They catch the model not performing the task at all, which is the failure a small model actually produces.

E. Unanswerable items and their taxonomy

The 80 unanswerable items are stratified into four classes, and the stratification is the point. A question about a topic absent from the corpus is rejected by any threshold and demonstrates nothing; reporting a single answerability F1 over a set dominated by such questions would conceal the system's behaviour on the cases that matter.

TABLE II — Unanswerable taxonomy (verified; design target in parentheses)

Class	n	Definition
Out-of-scope	25 (25)	Topic absent from the corpus entirely
Near-miss	29 (30)	Topic and scheme present; the specific fact is not
False premise	16 (15)	Presupposes a benefit or clause that does not exist
Under-specified	10 (10)	Unanswerable without naming a scheme

The two emphasised classes are the hallucination test. For both, retrieval returns a confidently scored, topically correct passage, and only a properly calibrated answerability signal abstains. These items are hand-authored rather than model-generated: a language model asked for a question its passage cannot answer reliably produces the trivial out-of-scope kind. Near-miss and false-premise items are additionally paired with the **distractor passage** they are designed to nearly match, so that an annotator can confirm the fact is truly absent rather than merely unfound.

F. Leakage control

Model-bootstrapped questions tend to reuse the distinctive wording of their source passage, which advantages lexical retrieval and would make any dense-versus-lexical comparison an artefact of annotation rather than a property of retrieval. The generation prompt instructs paraphrase rather than extraction, and we measure what that achieved.

Content-token Jaccard overlap, question against gold passage (verified set, $n = 313$):

Cell	n	median	max	leakage possible?
en→en	69	0.088	0.239	yes
hi→hi	59	0.094	0.245	yes

hinglish→en	51	0.036	0.129	yes
en→hi	45	0.000	0.107	no — cross-script
hi→en	44	0.015	0.088	no — cross-script
hinglish→hi	45	0.000	0.038	no — cross-script

The cross-script cells sit near zero because the two scripts share almost no tokens. That is the script boundary rather than evidence of annotation hygiene, so the figure that bears on leakage is the same-script one: **median 0.077, max 0.245** over 179 items, against 0.066 and 0.340 before verification. The model rewrites of §III-D did not make questions copy their passages; the one cell whose overlap rose, hinglish→en (median 0.009 to 0.036), rose because rewritten questions name their scheme in English, which is how such questions are actually typed.

Testing it requires holding the cell fixed. Our pre-registered plan was to repeat the retrieval comparison on the low-overlap tertile of the whole set, but the low tertile is then mostly one language pair, so the split measures the pair rather than the overlap. Within each same-script cell, splitting at the median overlap instead:

Cell	BM25 – dense, low half	high half	difference
en→en	+0.015	+0.000	−0.015
hi→hi	−0.017	+0.017	+0.034
hinglish→en	+0.060	+0.096	+0.036

No cell shows a lexical advantage that grows with overlap by more than a few points, and none of the within-half differences is significant (all $p \geq 0.25$). Leakage is not detectable in the verified set. Full output is in evals/report-leakage.txt.

IV. SYSTEM

A. Pipeline

The system is a conventional retrieval-augmented pipeline with one non-standard component. Documents are fetched and validated (§III-A, §III-B), normalized, segmented (§III-C), and indexed both lexically and densely. At query time the input is classified by language and script, retrieved against both indices, fused, passed to a generator constrained to the retrieved evidence, and gated by an answerability decision. Figure 1 gives the full diagram.

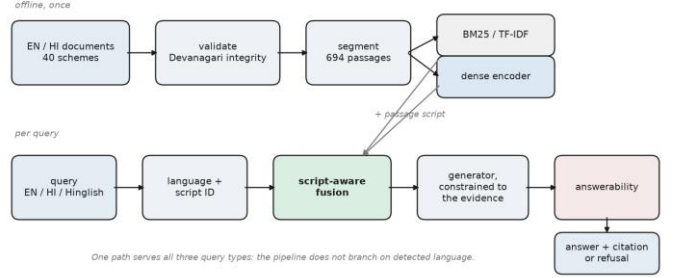


Fig. 1. System pipeline. Offline, once: Devanagari validation, segmentation, and lexical and dense indexing. Per query: language and script identification, script-aware fusion, generation constrained to the evidence, and an answerability decision. One path serves all three query types.

Crucially, **the pipeline does not branch on detected query language**. One path serves English, Hindi and code-mixed queries alike. This is deliberate: if retrieval varied by query type, the per-language comparison in §VI-B would measure the routing logic rather than the retriever.

B. Normalization

Normalization runs identically on passages and queries. This symmetry is the property that matters — a corpus normalized one way at index time and queries another at search time cease to match on exactly the cases of interest, and the resulting failure resembles poor model quality rather than a preprocessing bug.

Beyond Unicode NFC composition and whitespace repair, three corpus-specific equivalences are applied. Devanagari digits are mapped to ASCII, since a query written with one and a passage with the other otherwise never match. Monetary amounts are canonicalised, collapsing Rs. 3,50,000, ₹3.5 lakh, 3,50,000/- and ३,५०,००० to a single key; amounts are the most frequent answer type in this corpus and their surface form differs systematically between the English and Hindi versions of the same scheme. Sentence segmentation treats danda (।) and double danda (॥) as terminators alongside the full stop, without which a Hindi paragraph is returned as a single sentence — and normalization preserves both terminators, which is a precondition for that split rather than an incidental property, since segmentation normalizes a section before splitting it.

Where each equivalence actually applies. Two of the three act on the retrieval path; amount canonicalisation does not. It is called by answer scoring and error analysis, not by the index tokenizer, so at retrieval time ₹3.5 lakh and 3,50,000 share only the token 3. We state this because the paragraph above would otherwise imply a matching benefit that the system does not deliver, and because 87 of the 313 answerable verified items carry an amount in the question. Supplying the canonical key to the tokenizer as well moves BM25 Recall@5

on those 87 items from 0.640 [0.534, 0.736] to 0.674 [0.567, 0.766], a paired difference of +0.034 at $p = 0.25$, and +0.011 over all 313 ($p = 0.13$). The reason the gap costs so little is worth recording: the tokenizer splits on commas, so 3,50,000 already fragments into 3, 50 and 000, and the Devanagari form yields the same three subtokens once digits are mapped. Fragmentation supplies most of the equivalence incidentally, and the forms it does not cover — 3.5 lakh against 3,50,000 — are the rarer ones.

The symmetry claim itself does hold, and not only by construction: the indexing path normalizes a passage a second time, with soft-wrap joining disabled, while a query is normalized once. Applying the second pass changes the token sequence of 0 of 694 passages, so the two paths agree in practice as well as in intent.

Original surface text is retained alongside the normalized form throughout, so that citations display what the document actually said — a property §III-C records as having been broken in the committed corpus until it was repaired.

C. Query language identification

Queries are classified into the three classes the task requires — English, Indic, Code-Mixed — by a rule-based procedure. We prefer rules to a learned classifier here because the decision is defined by script and by a small closed set of function words, and because a misclassification changes the language the answer is generated in, making an explainable decision worth more than a marginal accuracy gain.

A query with a Devanagari character ratio above 0.60 is Indic. A query containing both Devanagari and Latin characters is Code-Mixed, tested by presence rather than ratio: a query such as *"PM-YASASVI के लिए eligibility criteria kya hai?"* is only 8% Devanagari by character count and would slip below any reasonable floor, yet is plainly code-mixed. A Latin-only query is Code-Mixed if it contains Hindi function words, and English otherwise.

The function-word lexicon carries the classification and requires one refinement. Several valid Romanizations of Hindi function words are also among the most frequent English words — *the* (थे), *is* (इस), *to* (तो), *us* (अस), *me* (मैं). Counting these as Hindi evidence classified plain English questions as code-mixed in development. We therefore partition the lexicon into unambiguous markers, which can trigger the code-mixed label, and ambiguous markers, which corroborate but never trigger it.

On the 400 gold questions the procedure agrees with the annotated language on every one: 143 English, 131 Indic, 126 Code-Mixed, no errors in either direction. We are careful about what that shows. These questions were generated per language-pair cell, so each is a clean instance of its class — a Hindi question is fully Devanagari, a Hinglish one is

consistently Romanized. Genuine user input is messier: transliteration is inconsistent, English and Hindi alternate within a clause, and typos cross the function-word lexicon. The result establishes that the rules implement the definition without a gap, not that the definition covers everything a user will type.

It matters mainly because everything downstream depends on it. The script the classifier reports is what decides which passages the lexical retriever is treated as eligible for (§IV-E), so a misclassification would disable the correction silently rather than raise anything.

D. Retrieval

Lexical. We implement TF-IDF with cosine scoring and BM25-Okapi ($k_1 = 1.2$, $b = 0.75$) over a script-aware tokenizer that dispatches per token rather than per document — necessary for code-mixed input, where English content words sit beside Romanized or Devanagari function words within one query. English tokens are casefolded and lightly stemmed; Hindi tokens receive light suffix stripping for common case markers. We note that a full Hindi morphological analyser is out of scope and that its absence costs some lexical recall on Hindi queries.

Dense. Five multilingual encoders are registered (Table III), of which four were evaluated; ai4bharat/indic-bert is a gated repository and could not be obtained, and we report its exclusion rather than omitting it silently.

TABLE III — Encoder registry

Model	Params	Type	Role
intfloat/multilingual-e5-base	278M	sentence	primary
paraphrase-multilingual-MiniLM-L12-v2	118M	sentence	speed baseline
LaBSE	471M	sentence	cross-lingual specialist
google/muril-base-cased	236M	MLM	Indic-pretrained
ai4bharat/indic-bert	33M	MLM	<i>gated, not evaluated</i>

MuRIL and IndicBERT are masked-language-model checkpoints, not sentence encoders: they were pretrained with an MLM objective and have no pooling layer trained for sentence representation. We include MuRIL because it is the model practitioners reach for by name on Indic tasks, and because the comparison isolates whether pretraining-language coverage substitutes for retrieval training. To prevent the result being dismissed as a pooling artefact we encode it under both mean and CLS pooling and report the better. Pooling is performed explicitly rather than delegated, which would otherwise silently attach an untrained mean-pooling head and present the result as a sentence encoder.

At 694 passages we perform **exact** search over an L2-normalized float32 matrix rather than using an approximate index. An ANN index at this scale would introduce recall error indistinguishable from model error, which this study cannot afford, since separating the two is its purpose. Every embedding cache stores a fingerprint of the passages it was built from and refuses to load when they differ — a silently misaligned embedding matrix produces plausible, ordered and meaningless results.

E. Fusion

We evaluate weighted score fusion, $\alpha \cdot \text{lex} + (1-\alpha) \cdot \text{dense}$ over min-max normalized scores, and Reciprocal Rank Fusion at $k = 60$. Both min-max and z-score normalization are implemented, and we report the ablation rather than asserting the preference: the two are indistinguishable on the gold set, $-0.003 \text{ Recall}@5$ for z-score at $p = 1.00$, and no point of the α sweep separates them by more than 0.013.

The reason usually given for preferring min-max — that BM25 scores are strongly right-skewed, so z-score leaves the fused ranking dominated by lexical outliers — holds in its premise and not in its consequence. BM25 candidate pools are right-skewed on 97.7% of queries (mean skewness 2.38), and the top candidate scores $2.86\times$ the pool mean against $1.06\times$ for the dense retriever. The fused ranking does not follow: z-score raises the lexical share of the top-1 fused score from 29.0% to 34.4%, and the share of top-5 slots held by passages the dense retriever never returned from 3.2% to 6.5%. The predicted direction is there; the magnitude never reaches the metric.

What separates the two normalizers is imputation, not skew. A passage returned by only one component contributes 0 from the other, and 0 is the floor of a min-max range but the mean of a z-scored one — an absent dense component ranks above 63% of the candidates it is compared against under z-score, and above none of them under min-max. We keep min-max because silence from a retriever should not be scored as an average opinion, which is a claim about what the fusion means rather than about what it measures. The ablation is `evals/report-fusion-normalisation.txt`.

RRF is left untuned to keep it an honest baseline against a tuned α .

We additionally propose **script-aware fusion**, which is the subject of §VI-E and is described there in full. In brief: a passage is scored over the retrievers *eligible* to return it, where a lexical retriever is ineligible for a passage whose script differs from the query's.

F. Generation and answerability

Generation is performed by a locally hosted quantized model behind a provider interface, with an extractive provider

available as a non-neural baseline and as a model-free fast path for testing.

Answerability is decided by one of four signals, evaluated independently so the paper can say which carries the decision. Two are **retrieval-side**: a threshold on the top score, and a calibrated logistic combination over the top score, the top-1-to-top-2 margin, mean top-k similarity, score spread and scheme agreement. Two read the **passage text**: the generator's own answerable report with its confidence, and a natural-language-inference check asking whether the cited passage entails the produced answer. All four are fitted on the development split by maximising F1 on the unanswerable class rather than accuracy, since the classes are deliberately imbalanced and accuracy is maximised by never abstaining.

The margin feature deserves note, because it is the one we expected to carry the retrieval-side signals. Maximum similarity indicates how good the best passage looks; it does not indicate whether anything else looked equally good. A near-miss unanswerable question should retrieve several passages at similar scores, because it matches a scheme's subject matter without matching any particular statement, whereas an answerable question usually has one clear winner — so a high maximum score with a small margin should be the signature of the hardest unanswerable class.

We state that as the design hypothesis it was, rather than as a property of the data. §VI-H measures it: the direction holds, and the magnitude does not. The split between the two kinds of signal is what the results turn on.

V. EXPERIMENTAL SETUP

A. Hardware, and why it is reported

All experiments run on an Intel Core i5-11320H (4 physical cores, 8 threads), 15.7 GB RAM, **with no GPU**. We report this not as a caveat but as a property of the work: every component was selected to run on a commodity laptop, and the full pipeline — corpus construction, five encoder sweeps, retrieval evaluation and generation — is reproducible without accelerator hardware. For work concerning access to public-information systems in India, reproducibility on the hardware students and civic technologists actually own is a relevant property.

The constraint shapes several choices: exact rather than approximate search, 4-bit quantized generation, and CPU-only PyTorch wheels.

B. Measured throughput

TABLE IV — Encoding time, 694 passages, 4 CPU threads

Encoder	Params	Time	Rate
---------	--------	------	------

MiniLM-L12	118M	149.6 s	4.6/s
multilingual-e5-base	278M	≈380 s	≈1.8/s
LaBSE	471M	≈600 s	≈1.2/s
MuRIL (mean pooling)	236M	632.3 s	1.1/s
MuRIL (CLS pooling)	236M	622.2 s	1.1/s

Two observations. First, the full registry encodes in under 40 minutes on CPU, which is the figure that makes the study reproducible. Second, **throughput does not track parameter count**: MuRIL at 236M is slower than LaBSE at 471M, because the MLM arm runs a hand-rolled encoding loop while the sentence-transformer models use an optimised batching implementation. Studies budgeting CPU inference from parameter counts alone will misestimate.

Retrieval latency is negligible by comparison: exact dense search over 694 passages is a single matrix-vector product, and BM25 scoring completes in a few milliseconds.

C. Splits and tuning protocol

The question set is partitioned into a **120-item development split** and a **test split** — 280 items as designed, 273 after verification removed 7 — stratified over query language, answerability and scheme, with seed 20260922. Stratification matters because an unstratified split leaves some cell of Table I with a handful of test items, and a per-language result computed on a handful of items reports noise with the appearance of authority.

Every threshold, fusion weight, retrieval depth and prompt is selected on the development split. The test split is scored once. We state this because the alternative is not recoverable after the fact: once a test figure has been seen, it cannot be unseen, and a protocol that permits iteration against it produces numbers that are optimistic by an unmeasurable margin.

The split was sealed on 2026-09-23 over the 393 verified items: **dev 120, test 273**, stratified by query language, answerability and scheme with largest-remainder allocation. The answerability threshold is fitted on dev and reported on test. The retrieval configuration — $\alpha = 0.4$, RRF $k = 60$, 50 candidates per component, the primary encoder — was fixed before the gold set existed and was not tuned on it, so retrieval is reported on test (Table in §VI-A) and, where the analysis needs the power, on dev and test together; the section says which. Figures from the earlier unverified set and from synthetic probes are marked as such where they are kept for comparison.

D. Metrics

Retrieval is scored with Recall@k, Precision@k and Hit Rate@k for $k \in \{1, 3, 5, 10\}$, Mean Reciprocal Rank, and

nDCG@10, with reciprocal rank taken as zero where no gold passage is returned. Question answering is scored with Exact Match and token-level F1 under a script-aware normalizer, plus semantic similarity against the gold answer. Answerability is scored with accuracy, precision, recall and F1 treating UNANSWERABLE as the positive class, with a confusion matrix and per-class recall.

The normalizer requires comment, because a normalizer written for English is wrong for Hindi in a way that does not announce itself. The widely used SQuAD normalization strips English articles, casefolds, and removes ASCII punctuation. Applied to Devanagari it strips articles Hindi does not have, casefolds to no effect, and leaves the danda — Hindi's sentence terminator, which is not ASCII punctuation — in place. The result is not an error but an Exact Match score a few points low on every Hindi row. Our normalizer dispatches on script and additionally unifies digit systems, monetary surface forms, and the joined and separated spellings of common Hindi postpositions.

E. Statistical protocol

Every headline figure carries a 95% confidence interval from a percentile bootstrap over queries with 1000 resamples. Comparisons between systems use a two-sided paired randomization test with 10000 trials, pairing on question identifier rather than position.

We use a permutation test rather than a t-test because per-query metric values are bounded, discrete and strongly non-normal — Recall@5 on a single question with one gold passage takes values in $\{0, 1\}$ — so a test assuming normality is inappropriate. The permutation test assumes only exchangeability under the null, which is precisely the statement that two systems are equivalent.

With a test split in the low hundreds, differences of two or three points are routinely within noise. We report intervals alongside every comparison because the alternative overstates the evidence at no saving.

VI. RESULTS

A. Headline results on the verified gold set

The corpus is 694 passages over 40 schemes in English and Hindi, with all 80 documents passing Devanagari integrity validation.

TABLE V — Recall@5 on the test split (216 answerable questions), with 95% bootstrap intervals

System	all	monolingual (83)	cross-lingual (62)	code-mixed (71)
BM25	0.581 [0.519, 0.646]	0.970	0.153	0.500

TF-IDF	0.593	0.964	0.185	0.514
multilingual-e5-base	0.720 [0.660, 0.775]	0.976	0.661	0.472
LaBSE	0.485	0.655	0.540	0.239
MiniLM-L12	0.478	0.624	0.605	0.195
MuRIL (mean pooling)	0.265	0.582	0.048	0.082
Hybrid, weighted $\alpha = 0.4$	0.688	0.976	0.484	0.528
Hybrid, plain RRF	0.618 [0.556, 0.681]	0.964	0.274	0.514
Hybrid, script-aware RRF	0.750 [0.692, 0.806]	0.964	0.685	0.556

Three findings carry the paper, and the verified set confirms the first two more cleanly than any earlier measurement and qualifies the third.

Lexical retrieval does not cross the language boundary. BM25 reaches 0.970 on monolingual questions and 0.153 on cross-lingual ones. MuRIL, pretrained on Indic text but not trained for retrieval, reaches 0.048. Figure 2 plots the breakdown.

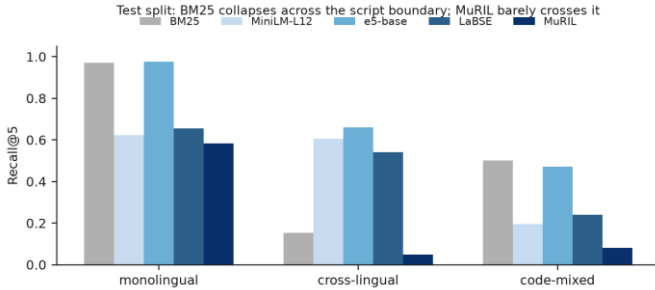


Fig. 2. Recall@5 by language group on the test split (216 answerable questions). Lexical retrieval collapses across the script boundary; MuRIL, pretrained on Indic text but not for retrieval, barely crosses it.

Plain fusion destroys the dense retriever's cross-lingual recall, and the script-aware correction restores it. Plain RRF over BM25 and e5-base reaches 0.274 on cross-lingual questions, against 0.661 for the dense retriever it contains: adding a lexical retriever that cannot see across the script boundary costs the hybrid system more than half of what its dense half finds. Script-aware fusion, which stops docking a passage for the lexical retriever's blindness to its script (§IV-E), recovers 0.685. Paired against plain RRF it gains +0.411 on the cross-lingual slice and +0.132 overall (both $p = 0.0001$, $n = 62$ and 216), and costs nothing monolingually (0.964 against 0.964). This is the result the method was designed around, and it is larger on the verified test split than on the probes (+0.131) or on the unverified set (+0.389 on the pooled items).

Against dense retrieval alone, the gain is small, and significant only on code-mixed questions. This is where the

verified set changes the story. Before verification, script-aware fusion beat e5-base alone by +0.133 on cross-lingual questions and +0.081 overall. On the test split it is +0.024 cross-lingually ($p = 0.62$) and +0.030 overall ($p = 0.052$); the only significant gain is on code-mixed questions, +0.085 ($p = 0.0037$), where Romanized questions carrying English scheme names give the lexical retriever something real to contribute. Pooling dev and test for power, the overall gain is +0.027 ($p = 0.021$). The margin shrank when verification rewrote 112 garbled questions, most of them in the cross-lingual and code-mixed cells; our reading is that fusion was partly compensating for questions a dense retriever could not match because they did not say what they asked, though the rewrite and the relabelling changed too much at once to isolate that cause. On well-formed questions, a strong multilingual dense retriever alone does nearly as well as the corrected hybrid. Figure 3 shows both comparisons on every slice.

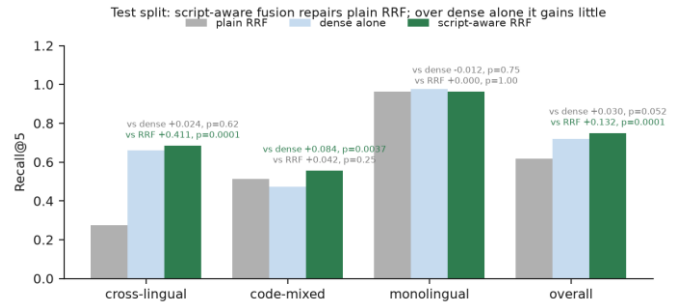


Fig. 3. Script-aware fusion on the test split, against plain RRF (the repair) and against dense retrieval alone (whether fusing is worth it), with paired randomization-test p-values.

So the claim this paper can make is specific. Hybrid retrieval assembled the standard way is actively harmful across scripts, the harm is structural, and a change of arithmetic removes it at no cost. Whether hybrid retrieval is worth assembling at all, over a good dense retriever alone, is a closer call than we reported before verification: it helps on code-mixed queries and is otherwise within noise.

Weighted fusion does not help at any weighting. Sweeping α over e5-base and BM25, no interior weighting beats pure dense on the test split (best $\alpha = 0.1$ at 0.720, equal to $\alpha = 0$; Figure 4), and none does on the pooled set (0.728 at $\alpha = 0.1$ against 0.736). H2, in its weighted-fusion form, is not supported. An earlier version of this section reported the opposite; that sweep used the wrong dense encoder for its endpoint, and was corrected before verification.

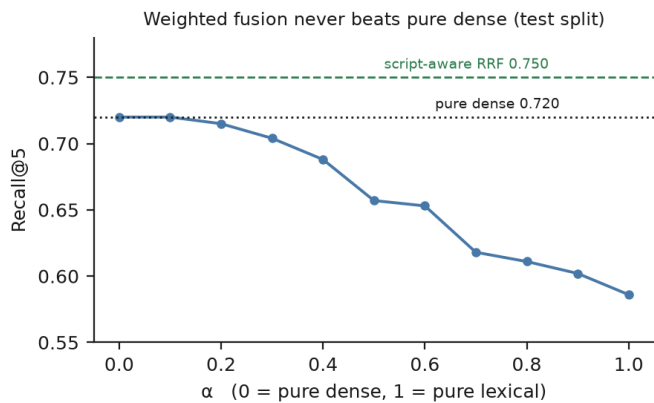


Fig. 4. Weighted fusion of BM25 and e5-base across α on the test split. No weighting beats pure dense; script-aware RRF, drawn for reference, does -- by changing how ranks combine rather than how much each retriever counts.

A.1 The correction is orthogonal to encoder choice

§VIII-E argues that the mechanism is about asymmetric coverage rather than about any particular encoder. Applying the same correction over each dense retriever (pooled dev and test, 313 questions):

Encoder	dense alone	with script-aware fusion	gain
multilingual-e5-base	0.736	0.764	+0.028
MiniLM-L12	0.504	0.666	+0.162
LaBSE	0.512	0.597	+0.085

Fusion improves every one, and the weaker the dense retriever the more the lexical half contributes once it is no longer allowed to penalise what it cannot see. Fusing over e5-base remains the best configuration: fusing over MiniLM is worse by 0.097 and over LaBSE by 0.167 overall (both $p \leq 0.0003$). MiniLM's fusion leads the cross-lingual slice (0.736 against 0.669) but not significantly ($p = 0.26$), and loses heavily on every other slice.

A.2 What the probe-set sections below do and do not show

§VI-B to §VI-F were measured on 180 synthetic probes, 120 of them monolingual and many lifting phrasing directly from their target passage, which flatters lexical retrieval. They are kept because they are where the fusion penalty was first found and diagnosed (§VIII), and the mechanism they describe — zero lexical recall across a script boundary, and a fusion rule that docks passages for it — is structural and is confirmed above. Their magnitudes are not the paper's results. Where they disagree with Table V, Table V stands; in particular §VI-F's observation that LaBSE leads the code-mixed slice does not hold on real questions (LaBSE 0.239 against 0.500 for BM25 and 0.556 for script-aware fusion).

B. Lexical against dense retrieval (Module 1)

Recall@5, overall, with 95% bootstrap confidence intervals [PROBE]

System	R@1	R@5	MRR	nDCG@10	R@5 95% CI
BM25	0.358	0.499	0.481	0.476	[0.429, 0.565]
TF-IDF	0.347	0.501	0.477	0.471	[0.430, 0.569]
MiniLM-L12 (multilingual)	0.122	0.303	0.280	0.282	[0.239, 0.366]
multilingual-e5-base	0.210	0.449	0.406	0.420	[0.386, 0.516]
LaBSE	0.175	0.361	0.361	0.341	[0.298, 0.433]
MuRIL (mean-pooled)	0.144	0.238	0.206	0.220	[0.178, 0.299]
MuRIL (CLS-pooled)	0.059	0.143	0.127	0.134	[0.099, 0.193]

BM25 leads overall, and the best dense encoder trails it by 0.049 ($p = 0.10$, not significant). We do **not** read this as evidence that lexical retrieval beats dense retrieval on this task. It is a property of the probe mix described in §VI-A — two thirds monolingual, many quoting the target passage — and the honest statement is that this comparison is not established either way on this data. The aggregate row is reported for completeness and the language-pair breakdown that follows is what carries the finding.

ai4bharat/indic-bert, named in the project brief, could not be evaluated: the repository is gated and returns HTTP 401 without an accepted licence.

C. The language-pair breakdown (Module 2)

The aggregate table conceals the result. Split by language group, the same systems behave in categorically different ways.

Recall@5 by language group [PROBE]

System	Monolingual (n=120)	Cross-lingual (n=30)	Code-mixed (n=30)
BM25	0.740	0.006	0.028
TF-IDF	0.745	0.006	0.019
MiniLM-L12	0.394	0.136	0.102
multilingual-e5-base	0.610	0.125	0.132
LaBSE	0.435	0.097	0.332
MuRIL (mean-pooled)	0.357	0.000	0.000

Lexical retrieval does not cross the language boundary at all: BM25 scores 0.740 monolingual and 0.006 cross-lingual — a factor of more than a hundred. The verified test split (Table V, Figure 2) has the same shape at less extreme magnitudes, 0.970 against 0.153, because real questions carry scheme names and numbers that occasionally survive the script change. This is not a weakness to be tuned away but the expected consequence of matching surface tokens between a Devanagari query and a Latin-script passage that share

essentially none. It is the empirical form of the structural claim in §VIII, and it is what makes the fusion behaviour in §VI-E a predictable consequence rather than a surprise.

D. Indic pretrained checkpoints underperform, with pooling controlled

MuRIL, pretrained on seventeen Indian languages, is the weakest dense encoder tested at 0.238 Recall@5. It sits below MiniLM-L12 at 0.303, a model with roughly half the parameters but trained with a retrieval objective — though we note that this particular ordering is **not statistically established**: paired over the 180 probes the gap is +0.065 at $p = 0.069$, which does not clear the 0.05 level we report against elsewhere. We ran both mean and CLS pooling and report the better, so the result is not a pooling artefact; CLS pooling is worse still at 0.143.

The sharpest form of the result needs no test at all, and is in the breakdown: MuRIL scores **exactly 0.000** on both the cross-lingual and the code-mixed slices — not a small number, but no correct retrieval in 60 queries. The model pretrained on seventeen Indian languages is the one that cannot retrieve between two of them.

The explanation is the known anisotropy of masked-language-model representations. An MLM checkpoint is trained to predict masked tokens, not to place paraphrases near one another, and its mean-pooled output occupies a narrow cone in which cosine similarity discriminates poorly. Pretraining-language coverage does not substitute for retrieval training. We report this because the brief names these checkpoints and because the assumption that an Indic-pretrained model is the right choice for an Indic retrieval task is a natural one to make.

E. Script-aware fusion (Module 3)

Plain Reciprocal Rank Fusion over BM25 and multilingual-e5-base scores 0.022 on cross-lingual queries — **below the 0.125 of the dense retriever it contains**. Fusion is actively destroying information on precisely the slice it was added to help. §VIII gives the mechanism; the correction is to score each candidate over the retrievers *eligible* to return it rather than those that did.

Recall@5, script-aware against plain RRF, paired randomization test (10 000 trials) [PROBE]

Slice	Plain RRF	Script-aware	Δ	p
cross-lingual	0.022	0.153	+0.131	0.0001 *
code-mixed	0.028	0.173	+0.145	0.0002 *
monolingual	0.712	0.668	-0.044	0.032 *
overall	0.483	0.500	+0.017	0.32

Figure 5 plots the table, with the monolingual regression on the same axes as the gains rather than in a separate panel.

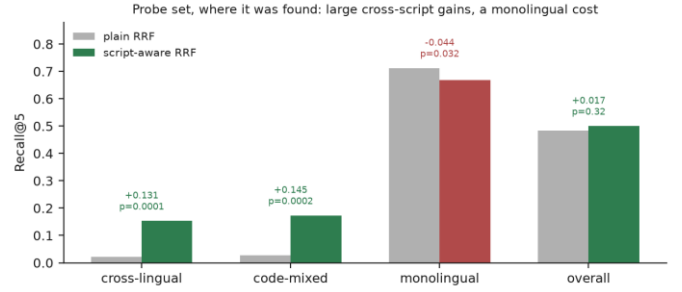


Fig. 5. [PROBE] Script-aware against plain RRF on the 180 synthetic probes, where the mechanism was found. The monolingual cost shown here does not appear on the verified set (Figure 3).

Against dense alone — the comparison that decides whether hybrid retrieval earns its place at all — script-aware fusion gains +0.028 cross-lingual ($p = 0.037$), +0.041 code-mixed ($p = 0.074$) and +0.050 overall ($p = 0.0086$).

Three things should be read from this table rather than one.

First, the gain on the target slices is large and significant. Second, the monolingual regression is real, significant, and the price of the method: removing the lexical double-vote costs precision where both retrievers could see the same passage and agreement between them genuinely was evidence. Third, the overall row is indistinguishable from noise, and a paper reporting only that row would conceal both the gain and the cost.

This also revises the negative result of the α sweep, which on the probes found no interior weighting significantly better than the better endpoint (the best, $\alpha = 0.4$, was +0.025 over pure lexical at $p = 0.13$) and recorded the hybrid hypothesis as unsupported. That verdict was correct about weighted fusion, and on the verified set no weighting beats pure dense at all (Figure 4). Whether hybrid retrieval beats its components once fusion stops penalising passages for their script is answered in §VI-A: it does on code-mixed queries, and is within noise elsewhere.

F. A caveat the fusion result does not remove

LaBSE, used alone, reaches **0.332** on the code-mixed slice against 0.173 for script-aware fusion over BM25 and e5-base. Paired over the 30 code-mixed queries that is +0.159 at $p = 0.0094$, so on code-mixed queries choosing a different encoder genuinely outperforms correcting the fusion over a weaker one.

The reverse holds elsewhere, and equally significantly. On the monolingual slice LaBSE loses to the fused system by 0.233 ($p = 0.0001$), and on the cross-lingual slice it is behind by 0.056, which is within noise ($p = 0.34$). LaBSE is not a

better retriever; it is a differently shaped one, and the shape happens to suit Romanized Hindi.

We state this plainly because it qualifies the practical reading of §VI-E. The fusion correction is orthogonal to the choice of encoder: it is a statement about how rankings are combined, and nothing prevents applying it over LaBSE. §VI-A.1 does so on the verified set: fusion over LaBSE gains +0.085 over LaBSE alone, and remains well below fusion over e5-base.

The result contradicts our own prior expectation, recorded before the experiment, that LaBSE would lead the cross-lingual slice by virtue of its translation-ranking objective. Instead it is the only model that handles Romanized Hindi well. On the probes that suggested selecting the encoder by query *script* rather than by language coverage. It did not survive the gold set (§VI-A.2): on verified code-mixed questions LaBSE reaches 0.239, against 0.500 for BM25 and 0.556 for script-aware fusion. We have not tested why the probes favoured it; the verification pass respelled the code-mixed questions, which changed their wording as well as their quality, so the two cannot be separated.

G. Direct LLM against RAG (Module 4)

We run four arms over a 72-item sample of the test split, stratified across the six language-pair cells: **A** closed-book, the model answering from its own parameters with no passages; **B** retrieval-augmented with dense retrieval; **C** retrieval-augmented with script-aware fusion; and **D** an oracle given the gold passage directly. The generator is Qwen2.5-3B-Instruct at 4-bit quantization throughout, so the arms differ only in the evidence they receive.

TABLE VI — Module 4 on the test split, 72 questions per arm

Arm	EM	token-F1 [95% CI]	abstains	Citation Support
A closed-book	0.000	0.050 [0.032, 0.070]	0.319	0.184
B RAG-dense	0.264	0.446 [0.351, 0.545]	0.250	0.870
C RAG-hybrid	0.236	0.418 [0.323, 0.517]	0.208	0.860
D oracle	0.403	0.641 [0.548, 0.731]	0.083	0.985

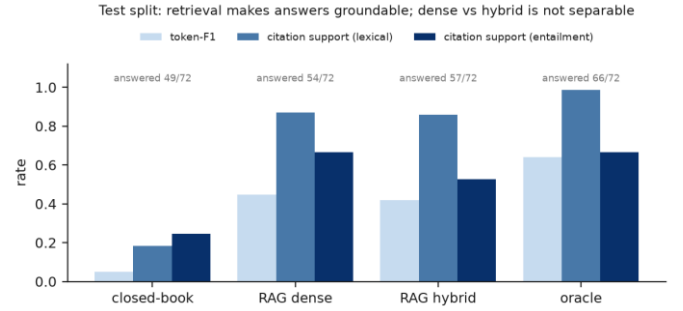


Fig. 6. Module 4 on the test split: token-F1 and Citation Support Rate, lexical and entailment variants, for each arm, with the number of the 72 questions each arm answered.

H3 is supported, and by a wide margin. Citation Support Rate — the share of answered questions whose answer the cited evidence supports — rises from 0.184 closed-book to 0.870 with dense retrieval, a paired +0.684 over the 38 questions both arms answered ($p = 0.0001$), and to 0.985 given the gold passage. The closed-book arm answers 49 of 72 questions and fewer than one in five of those answers is supported. It is not declining to answer; it recites scheme figures from memory and is usually wrong, and in the rare case it names a source at all, the source does not exist — it was shown no passages, so every identifier it produces is invented. With retrieval, fabricated citations fall to between 2% and 4% of answers. This is the behaviour the system exists to prevent, and retrieval prevents most of it.

Abstention falls as the evidence improves, from 0.319 closed-book to 0.083 for the oracle: the generator abstains less when it has something to work from.

The choice of retriever does not propagate to generation. Arm C against arm B is -0.027 token-F1 ($p = 0.55$) and $+0.000$ citation support ($p = 1.00$), paired over the questions both answered.

Comparison	token-F1	citation support
B vs A	+0.396, $p = 0.0001$ *	+0.684, $p = 0.0001$ *
C vs B	-0.027 , $p = 0.55$	+0.000, $p = 1.00$
D vs C	+0.223, $p = 0.0002$ *	+0.115, $p = 0.067$

This is expected rather than disappointing. On this sample the two retrievers differ mainly on code-mixed questions, where §VI-A found the only significant gap, and 72 questions spread over six cells leave 24 code-mixed ones — too few to carry a retrieval difference of +0.085 through a generator. Retrieval versus no retrieval is a large, significant effect on both metrics; *which* retriever, at this sample size, is not distinguishable.

G.1 Where the errors actually are

The oracle arm decomposes the remaining failure.

Quantity	Value
oracle (D) token-F1	0.641

full system (C) token-F1	0.418
retrieval error (D - C)	0.223
generation error (1 - D)	0.359

Given the correct passage and nothing to find, the 3B quantized generator reaches 0.641 token-F1. The generator still loses more than retrieval does, by a factor of 1.6, and the retrieval headroom of 0.223 is significant ($p = 0.0002$).

This revises what we reported before verification, where generation error was 0.597 against retrieval error of 0.193, a factor of three. The oracle arm is given the gold passage, so it measures how well the generator reads an item's evidence against its gold answer; on unverified items that included garbled questions and gold answers the passage did not state, and no generator scores well against either. The two samples also differ — this one is drawn from the test split — so we cannot say how much of the fall is the gold set's and how much the sample's. The direction is what matters for the reading: both components now lose comparable amounts, and a practitioner improving this system has a real choice rather than an obvious one.

G.2 Lexical and entailment support disagree

The two Citation Support variants rank the arms differently.

Arm	CSR lexical	CSR entailment
A closed-book	0.184	0.245
B RAG-dense	0.870	0.667
C RAG-hybrid	0.860	0.526
D oracle	0.985	0.667

H3 survives under both: every retrieval arm beats closed-book either way. But the retrieval arms lose between a fifth and two-fifths of their support when the criterion changes from "the answer's tokens appear in order in the cited passage" to "the passage entails the answer", and the entailment column puts arm C below arm B, as it did before verification. The oracle loses a third of its support under entailment while supported lexically at 0.985, which suggests the entailment model is stricter than the lexical check even on answers that copy their evidence, and that not every point it removes is a wrong answer. Whether it is well calibrated on Hindi and code-mixed answers we have not established, so we report both columns rather than choosing one.

Exact Match is low throughout (0.403 even for the oracle) and we do not read much into it. A 3B model asked an open question rarely reproduces a gold string verbatim, so EM here measures formatting agreement more than correctness; token-F1 is the informative column.

H. Answerability (Module 5)

A retrieval-score threshold is the standard abstention signal. On the verified set whether it works depends on which score it thresholds, and this reverses the finding we reported before verification.

All figures below fit the threshold, or the calibrated combination of §IV-F, on the 120-item dev split and report on the 273-item test split (57 unanswerable, a base rate of 0.21).

TABLE VII — Answerability on the test split, UNANSWERABLE as the positive class

Signal	score	test F1	precision	recall	abstains on
threshold	BM25 top score	0.525	0.408	0.737	38%
calibrated combination	BM25 features	0.481	0.431	0.544	26%
threshold	script-aware RRF top score	0.381	0.240	0.930	81%
calibrated combination	RRF features	0.454	0.435	0.474	23%

The lexical score separates the classes. Under BM25 the median top score is 25.5 for answerable questions and 13.8 for unanswerable ones, and every retrieval feature separates them: AUC 0.791 for the top score, 0.790 for the spread between the top and the k-th result, 0.766 for the mean of the top k, 0.717 for the top-1 to top-2 margin. A threshold fitted on dev catches every out-of-scope question on test, 0.857 of the under-specified ones and 0.667 of the false-premise ones, while answering 155 of 216 answerable questions. Figure 7 shows the whole trade-off across thresholds.

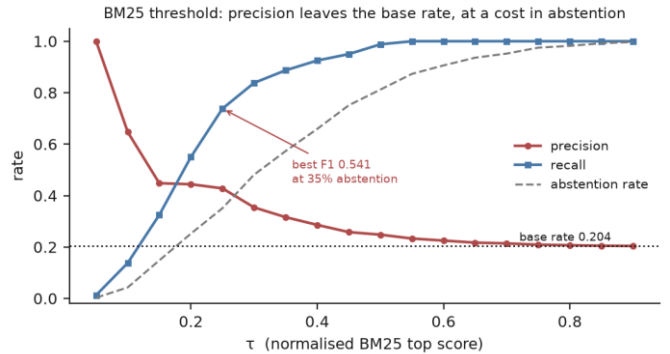


Fig. 7. BM25 top-score threshold swept over all 393 verified items, UNANSWERABLE as the positive class. Precision rises well above the 0.204 base rate; the best F1 is reached at 35% abstention.

It is weakest where it matters most. On near-miss questions — scheme present, specific fact absent — recall is 0.522: half of them retrieve as confidently as questions the corpus answers, because the passages they find are genuinely about the right scheme. A score threshold detects the absence of a topic; it detects the absence of a *fact* only about as often as

chance would suggest for a question drawn to sit right beside one.

Rank fusion discards the signal. The script-aware RRF score is built from ranks, not magnitudes, and its top score is 0.0328 for answerable questions against 0.0325 for unanswerable ones. Its best single feature reaches AUC 0.712, and a threshold fitted on dev lands at the degenerate end of the curve, refusing 81% of test questions for a precision of 0.240 — barely above the base rate. The hybrid retriever that §VI-A recommends for finding evidence is therefore the wrong place to read an abstention signal from; the lexical component's raw score, computed anyway as part of fusion, is the better signal by a wide margin.

Combining features does not beat the best one. The calibrated combination over BM25 features reaches AUC 0.781 against 0.791 for the top score alone, and a lower test F1 (0.481 against 0.525), though it abstains less. Its fitted weights put score spread first (1.245) and the top score second (0.636).

What changed, and why. Before verification we reported that no retrieval score separated the classes, that the median unanswerable question scored *higher* than the median answerable one under BM25, and that the top score was the least informative feature. All three were properties of the unverified questions. Of the 400 bootstrapped items, 85 did not name what they were asking about and 112 were unreadable; a question that does not name its scheme retrieves weakly whether or not the corpus answers it, which pushed answerable scores down onto the unanswerable ones. The leakage control of §III-F rules out the other explanation — that rewritten questions copied their passage's wording — since same-script overlap is essentially unchanged (median Jaccard 0.077 against 0.066) and no cell shows a larger BM25 edge on its high-overlap half.

We draw two conclusions. A lexical retrieval score is a usable first-stage abstention signal on well-formed questions: it removes out-of-scope and most under-specified questions cheaply. It is not an answerability signal for the near-miss class, which needs a model to read the passage (§VI-H.1), and it must be read from the lexical score rather than from a rank-fused one.

H.1 All four signals, on the same items

We evaluate all four on a 120-item sample enriched to hold every unanswerable item plus 40 answerable ones, fitting on its 35 dev-split items and reporting on its 85 test-split items. Enrichment is what makes the per-class table legible — a proportional sample of affordable size leaves two or three false-premise items in the reported half — and it moves the unanswerable base rate to 0.667, so the precision figures below are not comparable to the 0.21-base-rate figures in Table VII.

Signals 1 and 4 here read the script-aware RRF score, as the deployed system would; §VI-H showed that the BM25 score is the better retrieval-side signal.

TABLE VIII — Four answerability signals, 85 held-out items (57 unanswerable, 28 answerable)

Signal	reads	F1 (UNANS)	unanswerable caught	answerable answered	near-miss recall
1. retrieval threshold	scores	0.779	53 / 57	2 / 28	0.913
4. calibrated combination	scores	0.746	44 / 57	11 / 28	0.739
2. generator self-report	passage	0.889	52 / 57	20 / 28	0.957
3. NLI entailment	passage	0.889	52 / 57	20 / 28	0.957

Only the generator produces a system that answers questions. Signal 1 reaches recall 0.930 by refusing almost everything: it answers 2 of 28 answerable questions. Its high near-miss recall is the same refusal seen from the other side, not detection. The calibrated combination answers 11 of 28 and catches 44 of 57. The generator's own abstention answers 20 of 28 while still catching 52 of 57, and it is the only signal that is good on both sides of the matrix at once.

Per class, the generator catches every false-premise question (9 of 9), 22 of 23 near-misses and 17 of 18 out-of-scope ones. Its weak class is under-specified questions, 4 of 7, which we have not examined case by case. These rest on small counts, and the enriched sample has not removed that.

Before verification the same comparison had the generator answering 17 of 28 while catching 52 of 56, and the calibrated combination refusing every question. The ranking of the signals is unchanged. The generator and the calibrated combination now answer more of the answerable questions; the threshold on the fused score answers fewer (2 against 6), consistent with §VI-H's finding that rank fusion discards the score information a threshold would need.

H.2 Entailment adds nothing once the generator has abstained

Signals 2 and 3 are identical again, and for the same reason as before verification: both fits chose their degenerate threshold — `min_confidence` 0.00 and `τ` 0.000 — so each reduces to the same decision, whether the generator declined to answer. On the 85 held-out items the generator answers 25, of which 5 are unanswerable. With so little left to catch, no threshold on the model's self-reported confidence or on entailment improves F1, and the fit correctly declines to set one.

What can be said firmly is that on this corpus the generator's own abstention already does the work the

entailment check was added to do, and that a verification step has value in proportion to how much the generator lets through. A deployment would combine them in sequence rather than choose: the BM25 floor of §VI-H removes out-of-scope questions before any generation is spent on them, and the generator's abstention handles the near-misses the floor cannot see.

VII. ERROR ANALYSIS (MODULE 6)

A. Protocol

We analyse failures drawn against a fixed taxonomy of ten categories — code-mixing, transliteration variation, named entities, amounts, dates, ambiguity, near-duplicate schemes, the cross-lingual gap, long-tail schemes, and short queries — taking at most two cases per category.

Selection is by rule rather than by hand. Within each category we take the failures on which the system was most confident and most wrong, ranked by retrieval score against a failed gold match. This matters more than it may appear. Choosing illustrative cases after seeing which ones look interesting is how an error analysis becomes a collection of anecdotes: the examples then demonstrate whatever the author already believed. Fixing the taxonomy first, the count per category second, and the selection rule third means the cases are answerable to the taxonomy rather than to the argument.

Each case is recorded as a structured record — query, gold passages, the ranked output of every system, a diagnosis, and the component judged to have failed — and committed alongside its rendered report.

We ran the selection twice. The first run, twenty cases over plain RRF on the unverified questions (`evals/errors.jsonl`), is where the paper's main result came from, and §VII-C tells that story. Verification then rewrote every one of its twenty questions, so we repeated the selection on the sealed test split over the system the paper recommends, script-aware fusion (`evals/errors-test.jsonl`, `evals/report-errors-test.txt`). §VII-B reports that run. It yields 15 cases across 8 categories rather than 20 across 10: the near-duplicate and short-query categories produce no case under the rule, and we do not relax the rule to fill them.

B. What the verified cases show

The right scheme, the wrong passage. In 12 of the 15 cases script-aware fusion puts a passage from the correct scheme in its top five; what it misses is the one passage that states the fact. Often that passage is in the other language: a Hinglish question about how many documents a Sukanya Samridhhi account needs retrieves five English Sukanya passages, and the only statement of the answer is in the Hindi

article. The three remaining cases retrieve the wrong scheme altogether, one of them because the question spells Aadhaar as *adhar*.

Two failures are the gold set's, not the retriever's. The automatic diagnosis attributes all 15 cases to retrieval, and reading them corrects that twice. In E08 the retriever's first result states the answer — ₹17,000 crore set aside for Skill India in the 2017–18 budget — but the 25% overlap between adjacent passages copied that sentence into a passage the gold set does not list, so a correct retrieval is scored as a miss. In E04 the gold set omits the English passage that also states the 21-year maturity of a Sukanya account; that one remains a real miss, since the passage was not retrieved either. Verification added other-language passages to 73 items, and these two show it did not find every one. Overlap-duplicated sentences are a systematic source of such gaps, and resolving gold evidence to sentences rather than passages would close them.

One failure is a defect in script-aware fusion, and fixing it does not help. In E01 the top result is a Hindi passage about the Soil Health Card, returned for a Romanized question about the Antyodaya ration. BM25 reached that passage through the one Latin word it quotes, *Card*, and script-aware fusion then doubled the score, since for a Latin query a Devanagari passage counts as ineligible for a lexical vote even when it received one. Counting a cast vote as eligible is the obvious correction. Because the defect was found on the test split, we decided on dev: there the correction is worse, -0.026 Recall@5 overall and -0.093 cross-lingual, neither significant, and the test split moves the same way (`evals/report-observed-votes.txt`). Across both splits it changes six queries, losing five and gaining one; in four of the five losses BM25 had reached a cross-script gold passage through a year or number in the question, and in the fifth through a Latin acronym. A cast cross-script vote is usually real evidence, so the committed rule stays, and E01 is its price.

One failure is in the corpus. E06 asks which film based on the Swachh Bharat Mission was released in August 2023. The only passage stating it belongs to the *Beti Bachao* article, which mentions the film in passing, so a question that names Swachh Bharat retrieves Swachh Bharat passages. No retriever should be expected to find it.

The taxonomy tagger is not always right. E07 is filed under *amounts* because its question contains a year; it asks which movement inspired Atmanirbhar Bharat. The categories are assigned by heuristic, and a case's category is a pointer for reading it, not a finding.

That this sample is all retrieval is a property of the selection, not a finding about the generator. The cases are drawn from retrieval failures against a taxonomy of retrieval phenomena, so a generation error has no route into this sample.

§VI-G.1 makes the comparison the error analysis cannot: given the gold passage and nothing to find, the generator still loses more than retrieval does.

C. The case that produced the paper's main result

One case (E02 in evals/report-errors.txt) is worth stating in full, because the central contribution of this paper came out of it rather than out of a metric.

The aggregate tables showed hybrid fusion performing *worse* than dense retrieval alone on exactly the queries hybrid retrieval was supposed to help with, and gave no indication why. Both components were behaving as documented; the fusion was implemented correctly against its own specification; the α sweep found no interior setting significantly better than the better of its two endpoints, which reads as a clean negative result. Read as a table, the finding was that hybrid retrieval does not help here.

Reading the twenty cases gave the actual explanation. (They were drawn from the unverified questions, and verification has since rewritten all twenty; the mechanism they exposed is structural, and §VI-A confirms it on the verified set.) Of the twenty failures, the dense retriever alone had returned the gold passage in eight. Plain Reciprocal Rank Fusion lost all eight. A fusion method that discards every case its stronger component got right is not a method with a disappointing coefficient; it is a method whose arithmetic is wrong for this setting, and no amount of sweeping α would have revealed that, because α was never the problem.

The mechanism is developed in §VIII. The methodological point belongs here: the aggregate metric was not merely uninformative about the cause, it pointed away from it. Twenty cases, selected by rule and read in full, were what turned a negative result into a positive one.

VIII. DISCUSSION

A. Why fusion fails across a script boundary

Rank fusion rests on an assumption that is usually harmless and is false here. Combining two retrievers is worthwhile when their errors are complementary — when each recovers documents the other misses — and rank-based fusion in particular treats agreement between retrievers as evidence. A document returned by both systems accumulates two contributions; one returned by a single system accumulates one. Under Reciprocal Rank Fusion the fused score is $\sum_i 1/(k + \text{rank}_i)$, summed over the systems that returned the document, and the document found by one system alone is therefore scored at roughly half the document found by both.

Across a script boundary this arithmetic silently changes meaning. A lexical retriever matching surface tokens has, by

construction, *zero* recall over passages written in a script the query does not use: a Devanagari query and a Latin-script passage share essentially no tokens, so BM25 cannot return that passage at any rank, for any query, however relevant it is. The lexical retriever's silence about a cross-script passage is not evidence against that passage. It is not evidence at all.

Plain RRF cannot tell the difference between these two situations, because the sum is taken over the systems that *did* return the document rather than over the systems that *could*. A cross-script passage is docked a full contribution for the lexical retriever's blindness, and it is docked it relative to same-script passages whose lexical votes were possible. The penalty is therefore applied precisely where it is least deserved: the cross-lingual case is the one in which the dense retriever is the only informative signal, and it is the case in which fusion most aggressively discounts it.

It is worth being precise about what is and is not new. This behaviour is not a bug in RRF, which was designed for monolingual settings in which every retriever can reach every document, and under which discounting single-system evidence is sensible. Nor is the underlying difficulty unknown: work on multilingual retrieval has previously restricted sparse retrieval to same-language candidates when combining it with dense retrieval, which avoids the problem by not creating it. Our contribution is not the observation that sparse retrievers struggle across languages. It is the characterisation of *how the fusion arithmetic converts that limitation into an active penalty*, and a minimal correction stated in those terms.

B. Script-aware fusion

The correction follows directly from the diagnosis. We score each passage over the retrievers that were *eligible* to return it, rather than over those that did, treating a lexical retriever as ineligible for a passage whose script differs from the query's. A cross-script candidate is then judged on the dense contribution alone — the only evidence that ever existed about it — instead of being penalised for a vote that was never possible.

The change is small, requires no training and no tuning, and adds one lookup per candidate. On the sealed test split of the verified gold set it takes cross-lingual Recall@5 from 0.274 under plain RRF to 0.685 (paired randomization test, $p = 0.0001$, $n = 62$), and overall Recall@5 from 0.618 to 0.750 ($p = 0.0001$, $n = 216$). On the probe set where it was found, the same correction took cross-lingual recall from 0.022 to 0.153.

The comparison against dense retrieval alone is the one that decides whether hybrid retrieval earns its place at all, and here the verified set is less generous than the probes were. Script-aware fusion beats e5-base alone by +0.030 overall on the test split ($p = 0.052$) and +0.027 on dev and test pooled ($p =$

0.021); by slice, the gain is significant only on code-mixed queries (+0.085, $p = 0.0037$), where Romanized questions carrying English scheme names give the lexical retriever something the dense one misses, and it is +0.024 on cross-lingual queries ($p = 0.62$). The honest reading is that the correction removes a harm plain fusion introduces, and that once removed, fusing adds a small gain over a strong multilingual dense retriever, concentrated in code-mixed queries.

This also revises a negative result reported earlier in the same work, though less than we first claimed. The α sweep found no interior weighting that beat both endpoints, and still finds none on the verified set (best $\alpha = 0.1$ at 0.720 against 0.720 for pure dense, test split). We had read the script-aware result as showing that hybrid retrieval does beat its components once fusion stops penalising cross-script passages. On the verified set that holds for code-mixed queries and is within noise elsewhere.

C. The cost, stated plainly

On the probe set the correction was a trade: monolingual Recall@5 fell from 0.712 to 0.668, a regression of 0.044 at $p = 0.032$, because removing the lexical double-vote discards agreement between retrievers exactly where both could see the same passage and agreement really was evidence.

On the verified set there is no cost to state. Monolingual Recall@5 is 0.964 under both plain and script-aware fusion on the test split ($p = 1.00$), against 0.976 for dense alone ($p = 0.75$). The probe set's monolingual questions were built by lifting phrasing from their passages, which is the case where lexical agreement is most informative and discarding it costs most; real questions do not do that. We report the probe regression anyway, because on a deployment whose queries are overwhelmingly lexical-overlap monolingual ones it is the figure that would apply.

D. What the correction is worth end to end

The oracle arm of §VI-G puts a number on what retrieval can be worth end to end. Given the gold passage and nothing to find, our generator reaches 0.641 token-F1 on the test sample; the full system reaches 0.418. Retrieval error is therefore 0.223 and generation error 0.359. Both are substantial, and the generator still loses more, by a factor of 1.6.

We report this because it constrains the practical claim rather than the scientific one. Script-aware fusion is a large improvement to retrieval over plain fusion, measured as retrieval. It does **not** produce a measurable improvement in answer quality over dense retrieval alone: paired over the same 72 questions, arm C is 0.027 token-F1 below arm B ($p = 0.55$)

and level on citation support ($p = 1.00$). That is consistent with §VI-A, where the gain over dense alone is small and confined to code-mixed queries, of which the sample holds 24.

Before verification we reported generation error at 0.597 against retrieval error of 0.193 and advised fixing the generator first. On verified items the ratio has narrowed from three to 1.6, and that advice no longer follows. The ratio is a property of this configuration, not a general law — a 3B model at 4-bit quantization is a deliberately small generator, chosen so every number here is reproducible on commodity hardware, and a larger one would move the 0.359 without touching the 0.223. The point is that the two error sources have to be measured separately before either is optimised, which is what the oracle arm is for and why the brief asks for it.

E. Where this generalises

The mechanism is not about Hindi, and it is not about script. It appears wherever rank fusion is applied over retrievers with *asymmetric coverage* of the collection — where at least one retriever cannot, in principle, return part of what is being searched. Cross-script and cross-language pairs are the case studied here. The same structure arises when fusing a text retriever with one operating over images or tables, where each is blind to the other's modality; when fusing over access-controlled shards, where one retriever is permitted a subset of documents; and when fusing a domain-specialised index with a general one over a collection only partly covered by the specialist.

The practical guidance is correspondingly simple, and is the form in which we would want this result used: before fusing retrievers, check their *coverage*, not only their quality. Two retrievers of equal measured accuracy can combine well or destructively depending on whether each could have retrieved what the other did. Where coverage is asymmetric, fusion should be taken over the retrievers eligible for a candidate rather than those that returned it.

IX. LIMITATIONS

We state these at length, because the value of a result of this kind depends on knowing precisely how far it reaches.

The corpus is encyclopedic, not regulatory. Our documents describe Indian government schemes; they are not the scheme guidelines themselves. Sustained attempts to build the corpus from official portals failed on soft 404s and image-only scans — one eighteen-page document yielded seventeen characters of extractable text — and the deviation is documented in the product requirements rather than quietly absorbed. Consequently we make no claim about retrieval over statutory or regulatory Hindi, which differs from encyclopedic Hindi in register, sentence length and terminology. The cross-

script retrieval mechanism we identify is a property of the retrievers and the writing systems rather than of the genre, so we expect it to transfer; we have not shown that it does.

One Indic language. We evaluate Hindi and Hindi–English code-mixing. India has twenty-two scheduled languages, and several of the most widely spoken use scripts other than Devanagari. Script-aware fusion is formulated over an eligibility relation between queries and passages and is not specific to Devanagari, but a two-script corpus cannot distinguish a method that handles script asymmetry in general from one that happens to handle this pair. Behaviour with three or more scripts, and with partially overlapping scripts such as those sharing a Perso-Arabic base, is untested.

Scale, and one annotator. The evaluation set is 393 verified items, of which 313 are answerable, spread over six query-language-by-passage-language cells of 44 to 69 items, and the sealed test split holds 273 of them; per-cell differences of a few points are not resolvable. The larger limitation is how they were verified. The protocol called for a person to confirm every item and for a second person to re-label a blind sample, with $\kappa \geq 0.70$ as a gate on reporting any answerability result. The model passes did find and remove real defects — wrong answers, unreadable translations, duplicated unanswerable items — and a person then reviewed the whole test split (§III-D). But that review saw the model’s verdicts and accepted every item, so it confirms the model passes rather than measuring agreement with them; the dev split was checked by the model alone; and the blind second pass was a model instance, whose $\kappa = 1.000$ measures consistency, not the gate the protocol describes. An independent, blind second annotator remains the most valuable outstanding work.

Probe-set figures are kept, and they disagree with the gold set. The analyses in §VI-B to §VI-F were measured on 180 synthetic probes before the gold set existed, and are marked [PROBE]. The direction of the fusion result held on the verified set, as the mechanism predicted; the magnitudes did not. The probes understated the repair to plain fusion and overstated the gain over dense retrieval alone, and one probe-set observation — LaBSE leading the code-mixed slice — does not hold on real questions. Where they disagree, §VI-A stands.

A small quantized generator on CPU. Generation uses Qwen2.5-3B-Instruct at 4-bit quantization on a four-core CPU. This is a deliberate constraint — the system is intended to be reproducible on commodity hardware, and every figure we report is one a reader can regenerate without a GPU — but it bounds what the question-answering results say. A larger or unquantized model would plausibly change the absolute answer quality figures. It would not change the retrieval

results, which are model-independent, and the retrieval results are where our contribution lies.

The fusion correction is demonstrated over three encoders, not all of them. §VI-A.1 applies it over multilingual-e5-base, MiniLM-L12 and LaBSE on the verified set, and it improves each (+0.028, +0.162, +0.085), more for the weaker encoders, which is what a claim about the arithmetic rather than the representation predicts. That is evidence for the generalisation in §VIII-E but not proof of it: three sentence encoders on one corpus is a narrow base, and we have not tested the mechanism on the non-language asymmetries §VIII-E claims it reaches.

Most of our own pre-registered targets were missed. We fixed eight success criteria before collecting any number — retrieval recall by language group, answerability F1, citation support, annotator agreement — and met one of them: that the closed-book arm would sit far below the retrieval arms on citation support. We report this rather than quietly dropping the targets, and two of the misses are informative. The retrieval targets (0.85 monolingual, 0.70 cross-lingual) were set against an imagined corpus of official scheme guidelines, where a question and its answer share vocabulary; they were never recalibrated when that corpus proved unobtainable and an encyclopedic one was substituted, which is an error in our planning rather than in the system. The answerability target of 0.75 was unreachable by the signal it assumed, for the reason §VI-H gives. A target missed because the question changed under it is worth separating from a target missed because the method failed, and these are mostly the former.

No component is fine-tuned. All encoders and the generator are used off-the-shelf. Fine-tuning any of them on in-domain data would likely improve absolute numbers, and would confound the comparison we are making, which is between fusion methods holding the retrievers fixed.

The answerability threshold is a retrieval-side signal, and what it can detect is bounded by our taxonomy. On the verified set a BM25 top-score threshold is a usable first-stage filter (§VI-H), but its strength is uneven by class: it rejects every out-of-scope question on the test split and only 0.522 of near-misses. Since 55 of our 80 unanswerable items are written about schemes present in the corpus, the aggregate F1 depends on that mix. A benchmark whose unanswerable questions were predominantly out-of-scope would find the threshold far stronger, and one made entirely of near-misses would find it close to useless. We report per-class recall for that reason, and we evaluate the generator’s own abstention and an entailment check alongside it, since those read the passage and are the signals that can see a missing *fact* rather than a missing topic.

X. CONCLUSION

We have presented IndicRAG-QA, an evidence-grounded question answering system for Hindi, English and Hindi–English code-mixed queries over a parallel corpus of Indian government welfare schemes, together with a 393-item evaluation set across six query-language-by-passage-language cells and a taxonomy of four unanswerable question types, verified in two language-model passes with its test split reviewed by a person.

The central finding is about rank fusion rather than about any individual retriever. Reciprocal Rank Fusion, applied unmodified to a lexical and a dense retriever over a bilingual collection, penalises cross-script passages for the lexical retriever’s structural inability to return them, and in doing so discards the only informative evidence available in exactly the cross-lingual and code-mixed cases such a system exists to serve. Scoring each candidate over the retrievers *eligible* to return it, rather than those that did, is sufficient to correct this: on the verified test split cross-lingual Recall@5 rises from 0.274 to 0.685, with no measurable monolingual cost, no training and one additional lookup per candidate. Against a strong multilingual dense retriever used alone, the corrected hybrid gains little — +0.030 overall, significant only on code-mixed queries — so the finding is that standard fusion does harm and the correction removes it, not that fusion is a large win.

Two things about how the result was obtained seem worth carrying forward. The first is that it was invisible in the aggregate metrics, which showed only that hybrid retrieval underperformed, and was recovered from a structured reading of twenty failure cases; the α sweep that preceded it produced a clean negative result about the wrong question. The second is that the correction is a change of arithmetic rather than of model — it required no additional parameters, no training data, and no additional compute at query time — which suggests that the gap between the cross-lingual and monolingual performance of hybrid retrieval systems may be, in part, an artefact of how their components are combined rather than a limitation of the components themselves.

One measurement bounds all of this and belongs in the summary rather than only in the results. On the verified test sample, the oracle arm shows generation error of 0.359 against retrieval error of 0.223: correcting retrieval entirely would buy a little less than the generator is losing, and both are worth pursuing. Retrieval itself is what makes answers groundable — citation support rises from 0.184 closed-book to 0.870 — but which retriever supplies the passages did not show up in answer quality at our sample size. The fusion result is a claim about retrieval, measured as retrieval. Separating the two error sources is what the oracle arm exists to do, and on verified

items it changed our advice: before verification the generator looked like the only thing worth fixing, and it is not.

Immediate future work is an independent, blind second annotator for the evaluation set, to measure the agreement the protocol asks for; to extend the corpus beyond Hindi to at least one non-Devanagari Indic script, which would distinguish a general treatment of script asymmetry from a Hindi-specific one; and to test the eligibility formulation on asymmetric-coverage settings outside language entirely, of which multimodal and access-controlled retrieval are the most natural candidates.

REFERENCES

Verified against primary sources on 2026-09-21 unless marked otherwise.

#	Work	Venue
1	Cormack, Clarke, Bütcher. <i>Reciprocal rank fusion outperforms Condorcet and individual rank learning methods.</i>	SIGIR 2009, 758–759
2	Khanuja, Bansal, Mehtani, Khosla, Dey, Gopalan, Margam, Aggarwal, Nagipogu, Dave, Gupta, Gali, Subramanian, Talukdar. <i>MuRIL: Multilingual Representations for Indian Languages.</i>	arXiv:2103.10730, 2021
3	Li, Zhou, He, Wang, Yang, Li. <i>On the Sentence Embeddings from Pre-trained Language Models.</i>	EMNLP 2020, arXiv:2011.05864
4	Tu, Padmanabhan. <i>MIA 2022 Shared Task Submission: Leveraging Entity Representations, Dense-Sparse Hybrids, and Fusion-in-Decoder for Cross-Lingual QA.</i>	arXiv:2207.01940, 2022
5	Asai et al. <i>MIA 2022 Shared Task: Evaluating Cross-lingual Open-Retrieval QA for 16 Diverse Languages.</i>	arXiv:2207.00758
6	Zhang, Ma, Lin et al. <i>Mr. TyDi: A Multi-lingual Benchmark for Dense Retrieval.</i>	arXiv:2108.08787
7	Ogundepo et al. <i>AfriQA: Cross-lingual Open-Retrieval QA for African Languages.</i>	arXiv:2305.06897
8	<i>Exploring Anisotropy and Outliers in Multilingual Language Models for Cross-Lingual Semantic Sentence Similarity.</i>	arXiv:2306.00458
9	Feng, Yang, Cer, Arivazhagan, Wang. <i>Language-agnostic BERT Sentence Embedding.</i>	ACL 2022, 878–891
10	Wang et al. <i>Multilingual E5 Text Embeddings: A</i>	arXiv:2402.05672, 2024

	<i>Technical Report.</i>	
11	Reimers, Gurevych. <i>Sentence-BERT.</i>	EMNLP-IJCNLP 2019
12	Reimers, Gurevych. <i>Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation.</i>	EMNLP 2020, ACL Anthology 2020.emnlp-main.365
13	Kakwani et al. <i>IndicNLP Suite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Models for Indian Languages.</i>	Findings of EMNLP 2020, 2020.findings-emnlp.445
14	Rajpurkar, Jia, Liang. <i>Know What You Don't Know: Unanswerable Questions for SQuAD.</i>	ACL 2018, 784–789
15	Khattab, Zaharia. <i>ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT.</i>	SIGIR 2020, 39–48
16	Formal, Piwowarski, Clinchant. <i>SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking.</i>	SIGIR 2021 (short), arXiv:2107.05720